

Principles of Programming Languages

(Lecture 1)

COMP 3031, Fall 2025

Lionel Parreaux

Principles of Programming Languages 1

Principles of Programming Languages

Instructor

Lionel Parreaux (parreaux@ust.hk)

Teaching Assistants

- Yijia CHEN (ychenfo@connect.ust.hk)
- Heung Tung AU (htauac@connect.ust.hk)

The materials in this course were largely adapted from similar EPFL courses by Martin Odersky and Viktor Kuncak, most notably CS-210.

What is This Course About?

A tour of important programming language paradigms and constructs, with an emphasis on functional programming and Scala.

Goals of the Course (ambitiously)

To *expand your mind* and give you **new tools** enabling you to solve **hard problems** in **easier and better** ways.

Expected Learning Outcomes

Become better at modeling problems and using the right programming tools to solve them, using powerful programming language concepts.

Organization of the Course (1)

Lectures (L1)

Tuesday & Thursday, 12:00-13:20.

Rm G009B, CYT Bldg

Labs (LA1)

Mondays, 15:00-16:20

(Alternates between computer labs and pen & paper sessions.)

Rm 4214

Office Hours

Up to 40 min after the lectures.

Rm 3547

Midterm Exam

mid October (**TENTATIVE**)

Organization of the Course (2)

Assignments

There will be three programming assignments. (Details to be given later.)

Exams

There will be a midterm exam and a final exam. The final exam will cover all course material, with emphasis on post-midterm topics.

We will give you some material to prepare.

Grading

Each assignments counts for 10%; the midterm 30%; and the final 40%.

All assignments should be done independently by individual students. You can discuss among peers, but the hand-ins should be your own work.

No late hand-ins will be accepted.

Programming Paradigms

Paradigm: In science, a *paradigm* describes distinct concepts or thought patterns in some scientific discipline.

Main *programming* paradigms:

- imperative

Programming Paradigms

Paradigm: In science, a *paradigm* describes distinct concepts or thought patterns in some scientific discipline.

Main *programming* paradigms:

- imperative
- functional
- logic
- object-oriented
- concurrent
- parallel
- dependently-typed
- etc.

Review: Imperative Programming

Imperative programming is about

- modifying mutable variables and memory locations using assignments
- control structures such as if-then-else, loops, break, continue, return

The most common informal way to understand imperative programs is as instruction sequences for a von Neumann computer.

Imperative Programs and Computers

There's a strong correspondence between

Variables	≈	registers
Mutable fields	≈	memory cells
Variable dereferences	≈	load instructions
Variable assignments	≈	store instructions
Control structures	≈	jumps

Problem: Scaling up. How can we avoid conceptualizing programs word by word?

Reference: John Backus, Can Programming Be Liberated from the von. Neumann Style?, Turing Award Lecture 1978.

Scaling Up

In the end, pure imperative programming is limited by the “von Neumann” bottleneck.

One tends to conceptualize data structures word-by-word.

We need other techniques for defining high-level abstractions such as collections, polynomials, geometric shapes, strings, documents.

Ideally: Develop *theories* of collections, shapes, strings, ...

What is a Theory?

A theory consists of

- ▶ one or more *data types*
- ▶ *operations* on these types
- ▶ *laws* that describe the relationships between values and operations

Normally, a theory does not describe mutations!

Theories without Mutation

For instance the theory of polynomials defines the sum of two polynomials by laws such as:

$$(a * x + b) + (c * x + d) = (a + c) * x + (b + d)$$

But it does not define an operator to change a coefficient while keeping the polynomial the same!

Whereas in an imperative program one **can** write:

```
class Polynomial { double[] coefficient; }  
Polynomial p = ...;  
p.coefficient[0] = 42;
```

Theories without Mutation

Other example:

The theory of strings defines a concatenation operator ++ which is associative:

$$(a ++ b) ++ c = a ++ (b ++ c)$$

But it does not define an operator to change a sequence element while keeping the sequence the same!

(This one, some languages **do** get right; e.g. Java's strings are immutable)

Consequences for Programming

If we want to implement high-level concepts following their mathematical theories, there's no place for mutation.

- ▶ The theories do not admit it.
- ▶ Mutation can destroy useful laws in the theories.

Therefore, let's

- ▶ concentrate on defining theories for operators expressed as functions,
- ▶ avoid mutations,
- ▶ have powerful ways to abstract and compose functions.

Functional Programming

- ▶ In a *restricted* sense, functional programming (FP) means programming without mutable variables, assignments, loops, and other imperative control structures.
- ▶ In a *wider* sense, functional programming means focusing on the functions and immutable data.
- ▶ In particular, functions can be values that are produced, consumed, and composed.
- ▶ All this becomes easier in a functional language.

Functional Programming Languages

- ▶ In a *restricted* sense, a functional programming language is one which does not have mutable variables, assignments, or imperative control structures.
- ▶ In a *wider* sense, a functional programming language enables the construction of elegant programs that focus on functions and immutable data structures.
- ▶ In particular, functions in a FP language are first-class citizens. This means
 - they can be defined anywhere, including inside other functions
 - like any other value, they can be passed as parameters to functions and returned as results
 - as for other values, there exists a set of operators to compose functions

Some functional programming languages

In the restricted sense:

- Pure Lisp, XSLT, XPath, XQuery, FP
- Haskell (without UnsafePerformIO & related escape hatches)

In the wider sense:

- (Lisp, Scheme), Racket, Clojure
- SML, Ocaml, F#
- Haskell (full language)
- Scala
- (Smalltalk, Ruby, JavaScript)

(...): languages with first class functions but incomplete support for immutable data

History of FP languages

1959	(Lisp)	2003	Scala
1975-77	ML, FP, Scheme	2005	F#
1978	(Smalltalk)	2007	Clojure
1986	Standard ML	2012	Elixir
1990	Haskell, Erlang	2014	Swift
2000	OCaml	2017	Idris
		2020	Scala 3

Scala 3 is the language we use in this course.